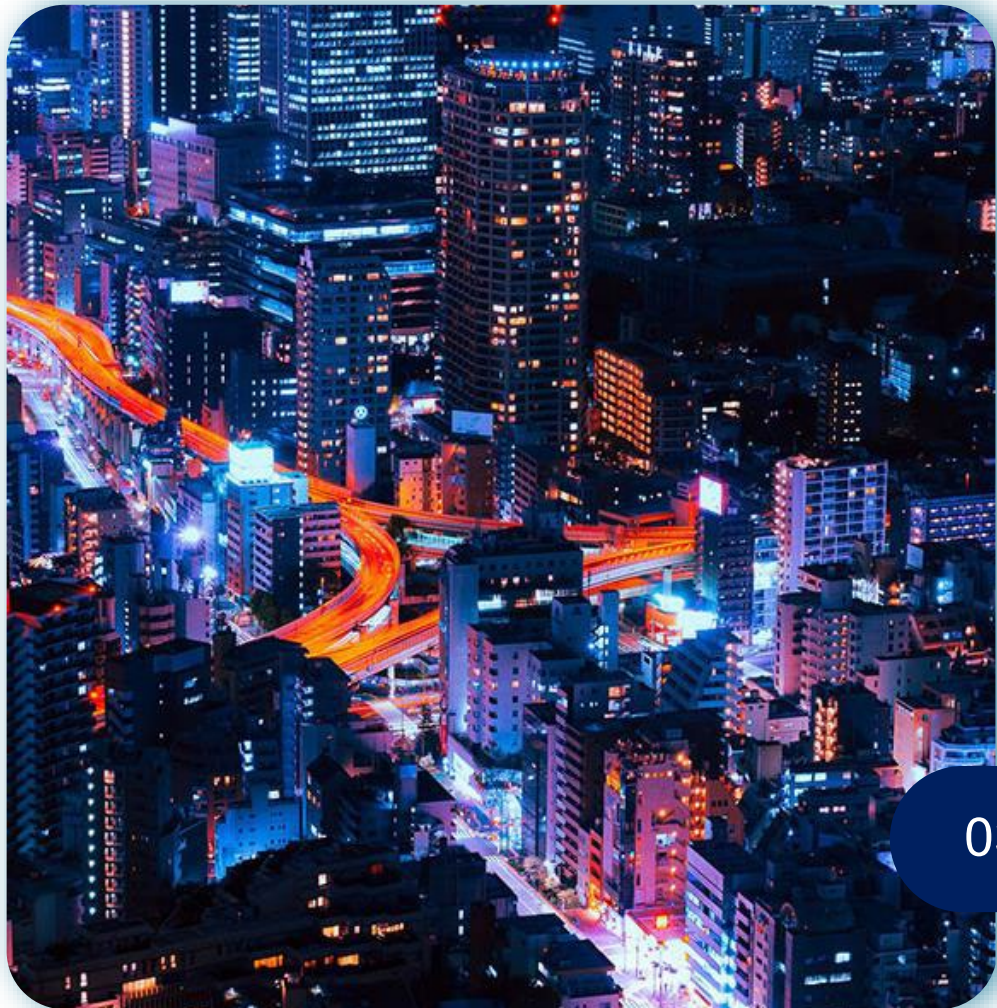




ML Fondamentaux

DSA 2026

UTHAYASOORIYAR Benno / LY Antoine

03/06/2026

Objectif et structure de la session

De l'apprentissage statistique au Machine Learning : comprendre ce qui change, ce qui reste, et ce qu'il faut maîtriser pour produire des modèles fiables en assurance

Bloc 1	De l'apprentissage statistique à l'apprentissage machine	1h45	<i>Stats et ML partagent un objectif mais diffèrent en philosophie</i>
Bloc 2	Biais-Variance et Généralisation	1h45	<i>Le cœur du ML = <u>généraliser</u>, pas fitter</i>
Bloc 3	Scikit-learn en pratique : Pipelines & Bonnes pratiques	1h45	<i>Sklearn, la bible du ML</i>
Bloc 4	Régression linéaire avec sklearn & Mise en perspective	1h45	<i>La régression linéaire contient déjà tous les enjeux du ML</i>

Ce que vous ne serez PAS à l'issue de ce cours

- Experts en Python
- Experts en Machine Learning / Deep Learning
- Capables de deployer en production un modèle (sans formation complémentaire)

*Mais vous aurez les **fondations** solides et un **recul** critique pour aborder n'importe quel projet d'apprentissage machine.*

Pré-requis

- Python 3.9+
- Jupyter Notebook / Google Colab

```
pip install numpy pandas matplotlib seaborn scikit-learn
```

Bloc 1

De l'apprentissage statistique
à l'apprentissage machine

Apprentissage statistique

L'essor du ML : Pourquoi ?

Explosion des données, puissances de calcul, écosystèmes open-sources: tout les éléments sont disponibles pour construire des outils plus puissants.

Et en assurance ?

Triage automatique de sinistres, risque climatique, automatisation de la chaîne de souscription...

Apprentissage statistique

*Comprendre un phénomène avec des hypothèses
sur la donnée
(effets, causes)*

? Quel est l'impact de l'âge sur la fréquence de
sinistre?

Apprentissage machine

*Prédire un phénomène avec peu d'hypothèses sur la
donnée
(prédire au mieux)*

? Quel modèle prédit le mieux les sinistres ?

*"There are two cultures in the use of statistical modeling to reach conclusions from data.
One assumes that the data are generated by a given stochastic data model.
The other uses algorithmic models and treats the data mechanism as unknown."*

Apprentissage statistique

Le paradigme statistique

Données : n observations $(Y_i, X_i) \in \{0, 1\} \times \mathbb{R}^p$ de distribution $\mathbb{P}$ inconnue

Paradigme [Vapnik, 1998] — 3 piliers :

1. **Hypothèse de forme** : Les fonctions linéaires en leurs paramètres (avec $p \ll n$) peuvent bien approximer la relation entre x_i et y_i
2. **Hypothèse de distribution** : La loi statistique sous-jacente est connue (*normale, Poisson, etc.*)
3. **Outil d'estimation** : La maximisation de la vraisemblance est un bon outil pour estimer les paramètres

Motivation principale : l'INFÉRENCE (comprendre et expliquer Y en fonction de X)

Illustration : Le GLM

Modèle statistique $(\Omega, \mathcal{F}, \{\mathbb{P}_{\theta, \phi}\}_{\theta \in \Theta, \phi > 0})$ dit exponentiel si :

$$f_{\theta, \phi}(y) = \exp \left(\frac{y\theta - b(\theta)}{a(\phi)} + c_{\phi}(y) \right)$$

Avec des **hypothèses**:

1. $(Y|X = x) \sim \mathbb{P}_{\{\theta(x), \phi\}}$ (hypothèse distributionnelle)
2. $\mathbb{E}[Y | X = x] = g^{-1}(W^T x)$ avec $W \in \mathbb{R}^p$ estimé par maximum de vraisemblance

→ **Hypothèse distributionnelle contraignante** :

la réalité ne suit pas toujours une loi connue

→ **Linéarité/Hypothèse de forme**:

les relations complexes (interactions, non-linéarités) sont difficiles à capturer

→ **Focus inférence** :

la capacité de généralisation n'est pas la motivation principale [Breiman, 2001]

→ **Dimension** :

quand p est grand ou que les données sont non-structurées (texte, images), les GLM atteignent leurs limites

Apprentissage statistique

Exemple : la régression logistique vue des deux côtés

$$\mathbb{P}(Y_i = 1|X_i = x_i) = \frac{e^{\beta^T X_i}}{1 + e^{\beta^T X_i}} = \frac{1}{1 + e^{-\beta^T X_i}}$$

	Vision Statistique	Vision ML
Hypothèse	$X \sim \mathcal{B}(n, p)$	Y
Estimation	Max. vraisemblance	Min. risque empirique (fonction de perte)
Validation	Tests stats, p-value	Cross-validation, AUC
Objectif	Odds ratios, interprétation	Score prédictif, généralisation
Résultat	<i>“L’âge augmente le risque de 12% ($p<0.05$)”</i>	<i>“Le modèle prédit le sinistre avec une AUC de 0.82”</i>

Vers l'Apprentissage Machine

Relâcher l'hypothèse distributionnelle

Cadre formel :

- On suppose qu'il existe $\mathbb{P}_X$ telle que $X \sim \mathbb{P}_X$ (qu'on n'explicite pas nécessairement)
- On suppose qu'il existe une fonction cible $f \in \mathcal{Y}^X$ (qu'on ne connaît pas) telle que $Y = f(X) + \epsilon$

L'apprentissage machine consiste à trouver un proxy m de f en minimisant le risque empirique :

$$\mathcal{R}_{\mathbb{P}_X, f}(m) = \mathbb{P}_X(m(X) \neq f(X))$$

Comme en statistique, on utilise un jeu de données $(x_i, y_i)_{i \in \llbracket 1, n \rrbracket}$ pour estimer le risque empirique (et le minimiser). Celui-ci dépend d'une distance l entre les données et les prédictions du modèle:

$$\hat{\mathcal{R}}_n(m) = \frac{1}{n} \sum_{i=1}^n l(m(x_i), y_i)$$

Sans hypothèse de distribution, comment savoir si l'estimateur est proche de la réalité ?

→ Méthodes de validation (cross-validation, train/test split)

Vers l'Apprentissage Machine

Les nouvelles questions que pose le ML

Quelle famille de fonctions pour l'estimateur m ?

Arbre, réseau de neurones, linéaires, etc.

Quelle fonction de perte l ?

MSE, log-loss, etc.

Comment optimiser ?

Descente de gradient, solveur, etc.

Comment évaluer sans hypothèse ?

Cross-validation, test set

Combien de données faut-il ?

Réponse avec la courbe d'apprentissage

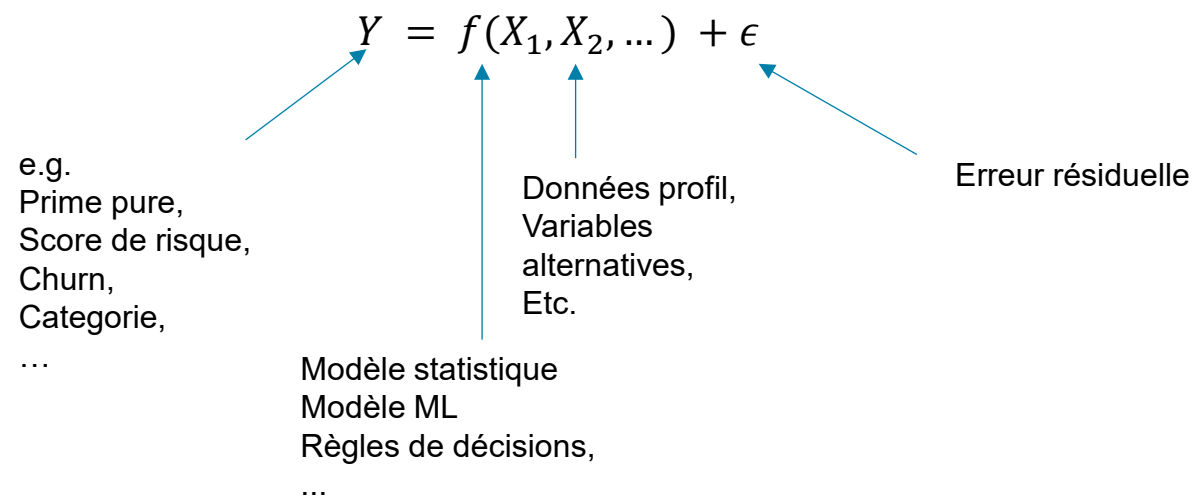
Quand s'arrêter ?

Early stopping, régularisation

Vers l'Apprentissage Machine

Lien avec l'assurance

La structure fondamentale reste la même:



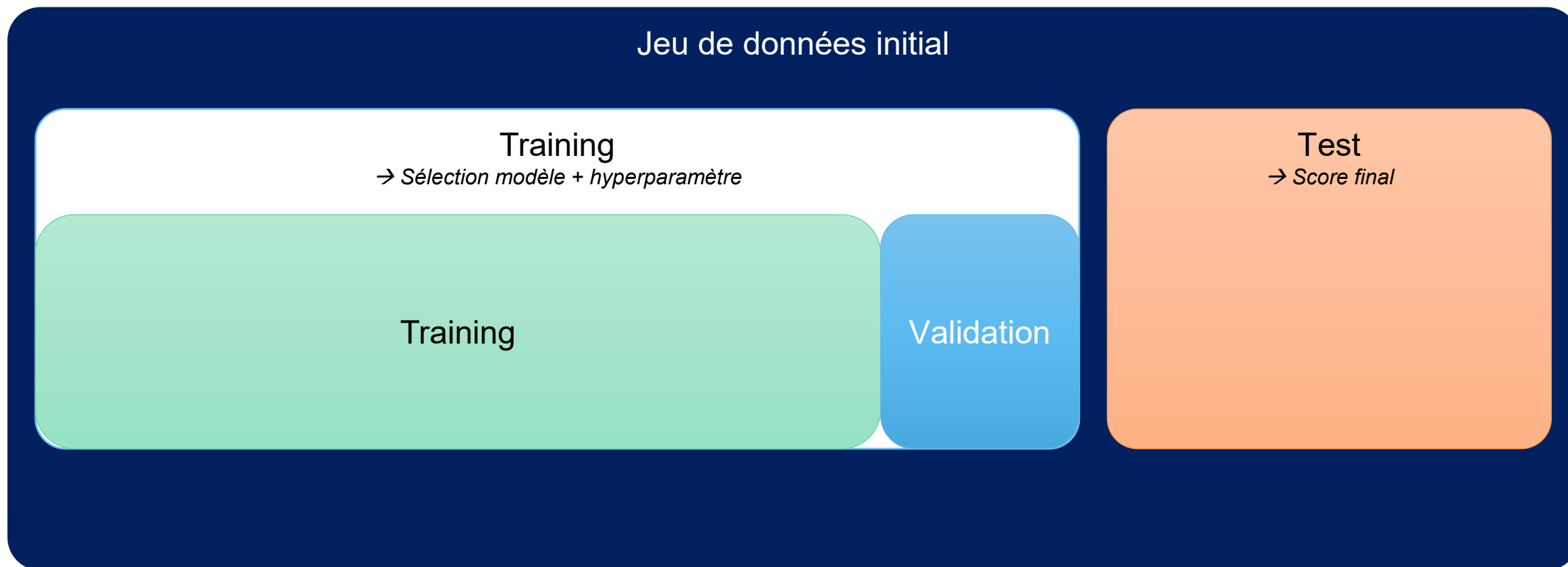
En statistique, ϵ vient d'une hypothèse distributionnelle.

En ML, ϵ doit être évalué **empiriquement** (d'où l'importance de la validation).

Comment valider efficacement ?

Vers l'Apprentissage Machine

Le schéma de validation en ML



Vers l'Apprentissage Machine

Tableau synthétique

	Statistique classique	Machine Learning
Objectif principal	Comprendre, inférer, expliquer	Prédire, généraliser
Hypothèses	Distribution connue (Poisson, Gaussienne...)	Aucune (ou faible)
Estimation	Maximum de vraisemblance	Minimisation du risque empirique
Validation	Tests statistiques (p-value, AIC, BIC)	Cross-validation, hold-out
Interprétation	Native (coefficients, odds ratios)	À construire (SHAP, LIME, permutation importance)
Complexité du modèle	Faible (linéaire, paramétrique)	Variable (de linéaire à réseau profond)
Taille de données	Fonctionne avec peu de données	Souvent besoin de plus de données
Risque principal	Mauvaise spécification du modèle	Overfitting
Exemple en assurance	GLM Poisson pour la fréquence	GBM pour la fréquence
Outil typique	R, SAS, statsmodels	scikit-learn, XGBoost, PyTorch

Vers l'Apprentissage Machine

Quand utiliser quoi ?

Mon problème nécessite-t-il ...

... d'expliquer au régulateur / auditeur ?

→ *Modèle statistique (GLM) ou ML interprétable (arbre, GAM)*

... la meilleure performance predictive ?

→ *ML*

... les deux ?

→ *ML + Outils d'explicabilités, ML interprétable, approche en deux couches*

... peu de données (<1000 observations) ?

→ *Statistique Classique (plus robuste avec peu d'observations)*

Bloc 2

Les paradigmes d'apprentissage

Paradigmes d'apprentissage en ML

Lexique des approches

Paradigmes ML / Méthodes d'apprentissage

Apprentissage
supervisé

Apprentissage
non-supervisé

Apprentissage par
renforcement

Hypothèses de relations

Prédicteurs linéaires

Prédicteurs non-linéaires

Nature des données

Catégorielles
(Classification)

Numériques
(Régression)

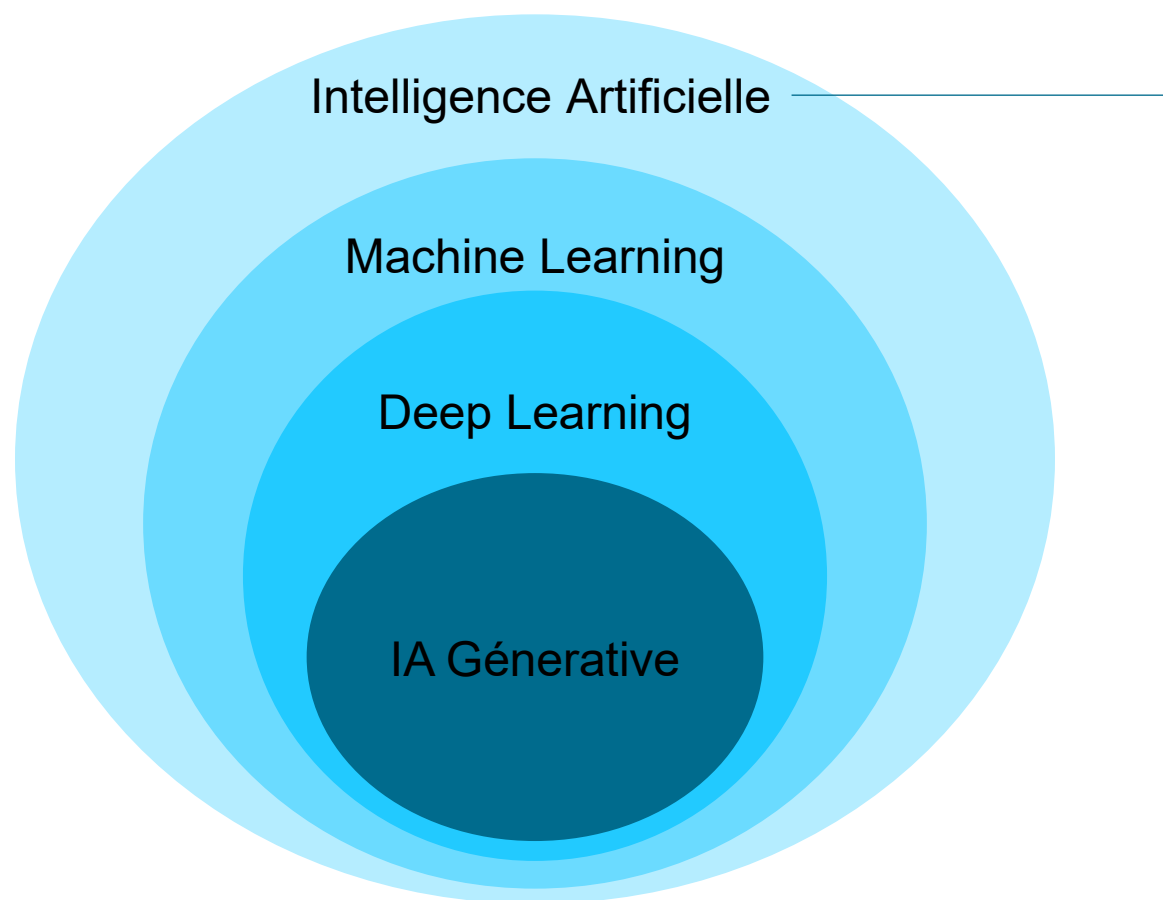
Approches de modélisation

Générative

Discriminative

Paradigmes d'apprentissage en ML

Lexique des approches

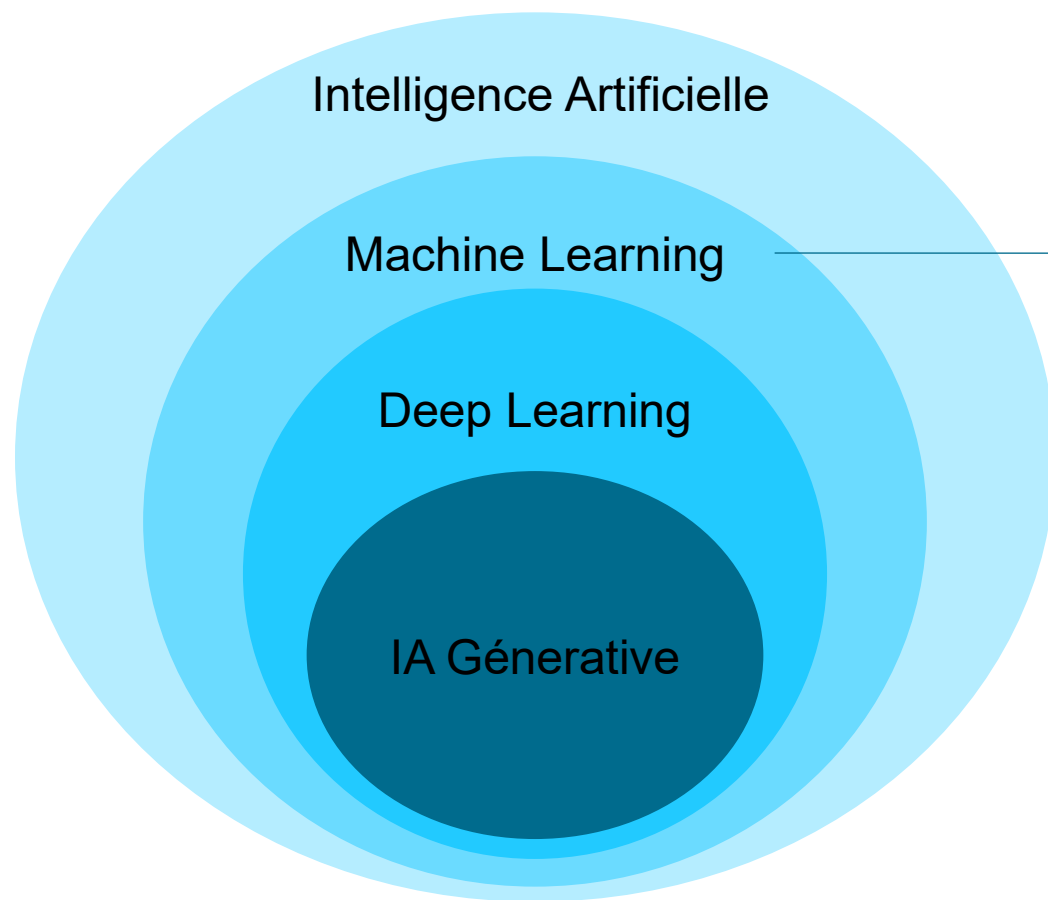


Concevoir un système faisant preuve d'intelligence :

- Trier des pièces selon leurs caractéristiques visuelles (taille, couleur, forme) dans une usine
- Jouer aux échecs
- Se déplacer d'un point A à un point B dans un supermarché bondé
- Appliquer les règles de souscription pour déterminer la prime d'assurance d'une personne assurée

Paradigmes d'apprentissage en ML

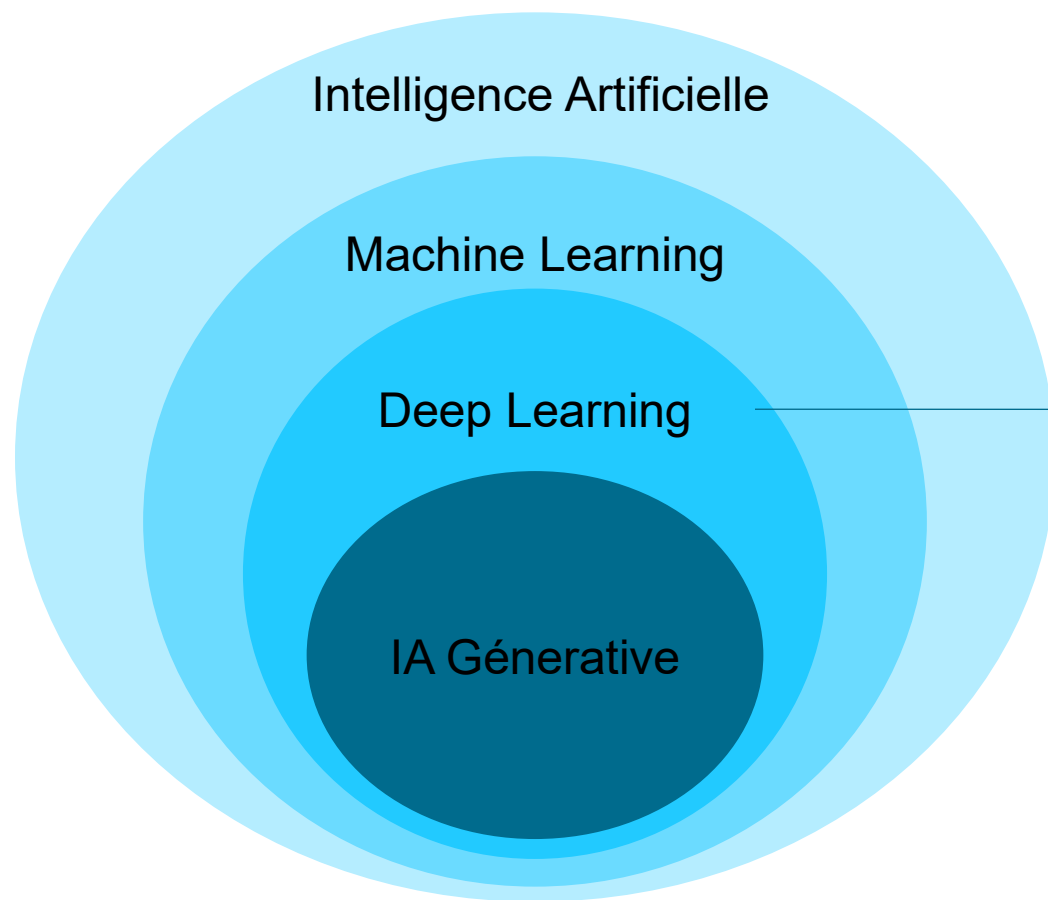
Lexique des approches



Concevoir un système capable de reproduire un comportement intelligent
+ Données sur le comportement à adopter

Paradigmes d'apprentissage en ML

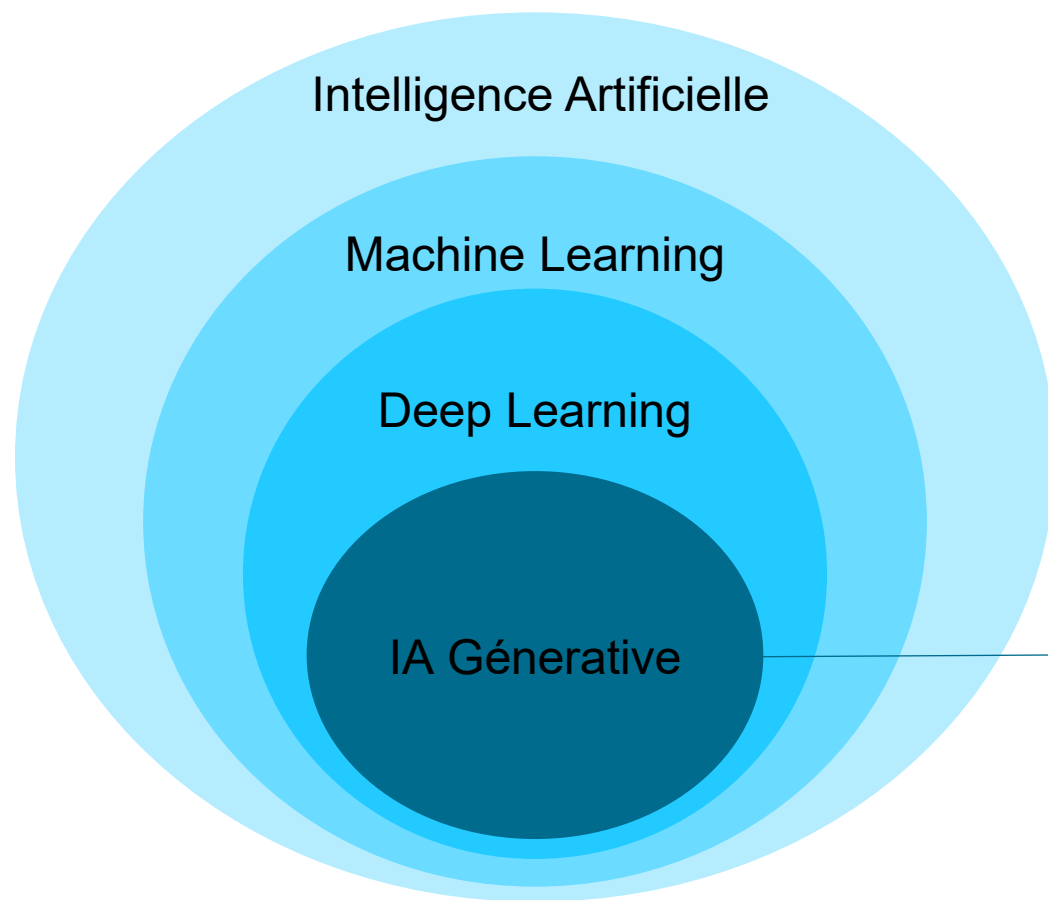
Lexique des approches



Concevoir un système capable de reproduire un comportement intelligent
+ Données sur le comportement à adopter
+ Architecture neuronale

Paradigmes d'apprentissage en ML

Lexique des approches



Concevoir un système capable d'afficher un comportement intelligent

+ Données sur le comportement à adopter

+ Architecture neuronale générant du contenu (basée sur le modèle Transformer)

Exercise

Biais & Variance

Overfitting

Analogie pédagogique



Etudiant A : Apprend par cœur le chapitre 1 d'un cours.

Jour du partiel : il répond parfaitement à l'exercice sur le chapitre 1, mais se trompe sur tout le reste

→ *sous-apprentissage*



Etudiant B : Apprend par cœur tous les exercices corrigés du TD.

Jour du partiel : Il est perdu car les valeurs numériques ont changé. Il a mémorisé les réponses, mais il ne sait pas généraliser.

→ *sur-apprentissage*



Etudiant C : Il a appris le cours, il s'est entraîné sur le TD, mais il n'a pas appris par cœur. Il comprend les principes sous-jacents, et peut les appliquer à des exercices qu'il n'a jamais vus.

→ *Apprentissage idéal*

Overfitting

Analogie pédagogique

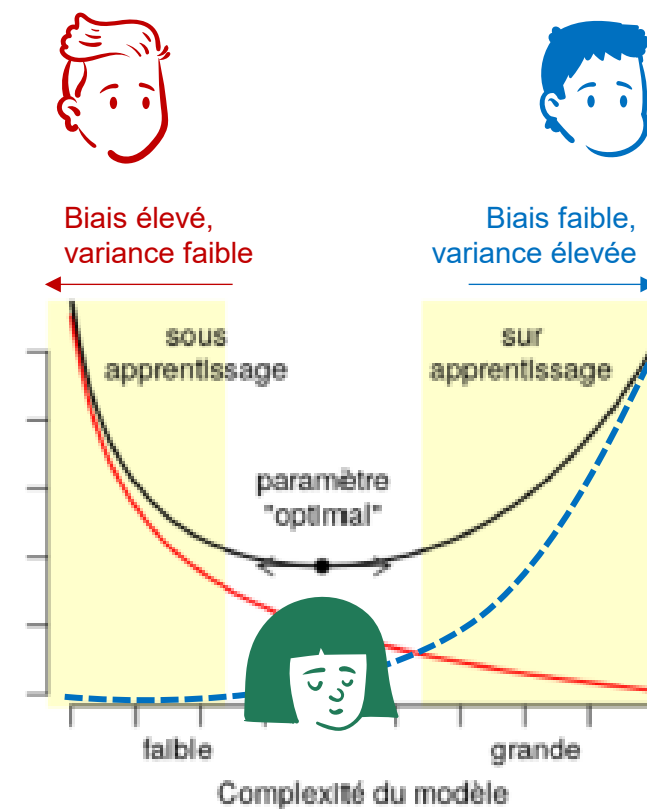
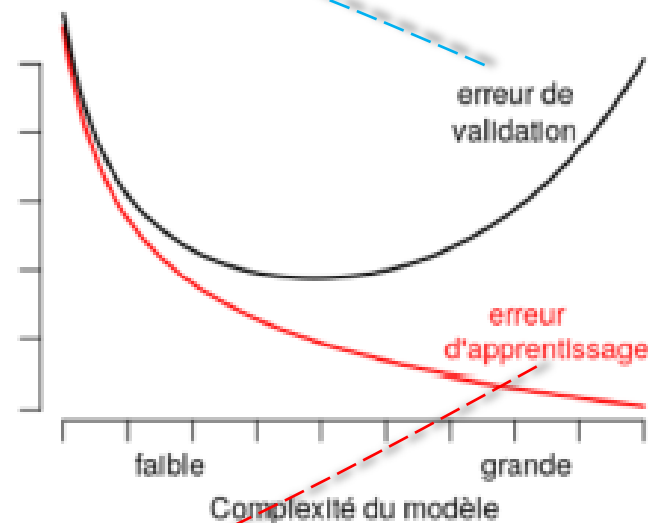
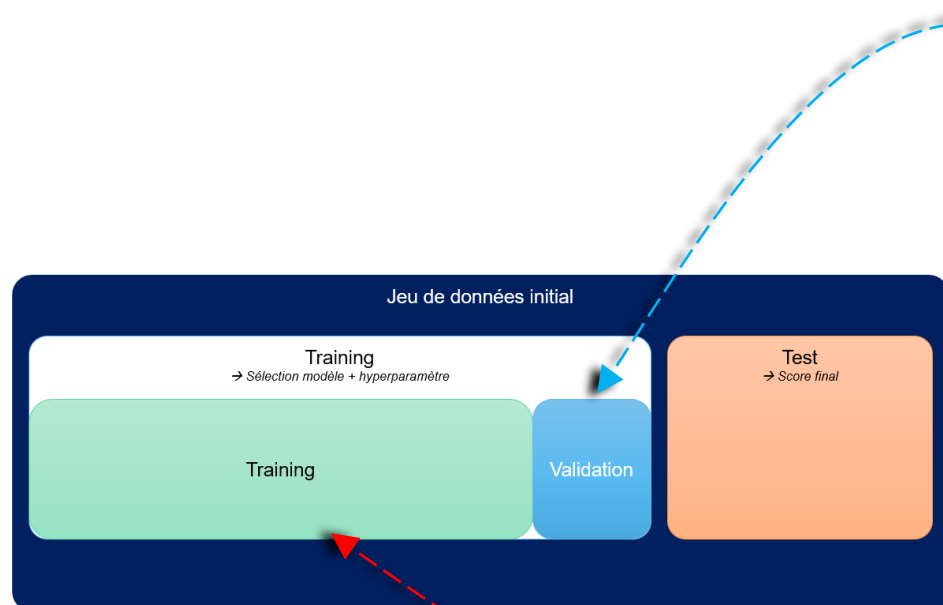


Figure 51.: Généralisation, et sur-apprentissage.

- **Biais :**
Erreur systématique due à des hypothèses trop simplistes
- **Variance:**
Sensibilité excessive aux fluctuations du jeu de données d'entraînement

Overfitting

Exemples en assurance

Tarification sur-apprise:

→ *Le modèle colle aux fluctuations historiques, mais les tarifs sont instables d'une année sur l'autre*

Scoring fraude sur-appris:

→ *Détecte les patterns du passé mais rate les nouveaux schémas de fraude*

Provisionnement sur-appris:

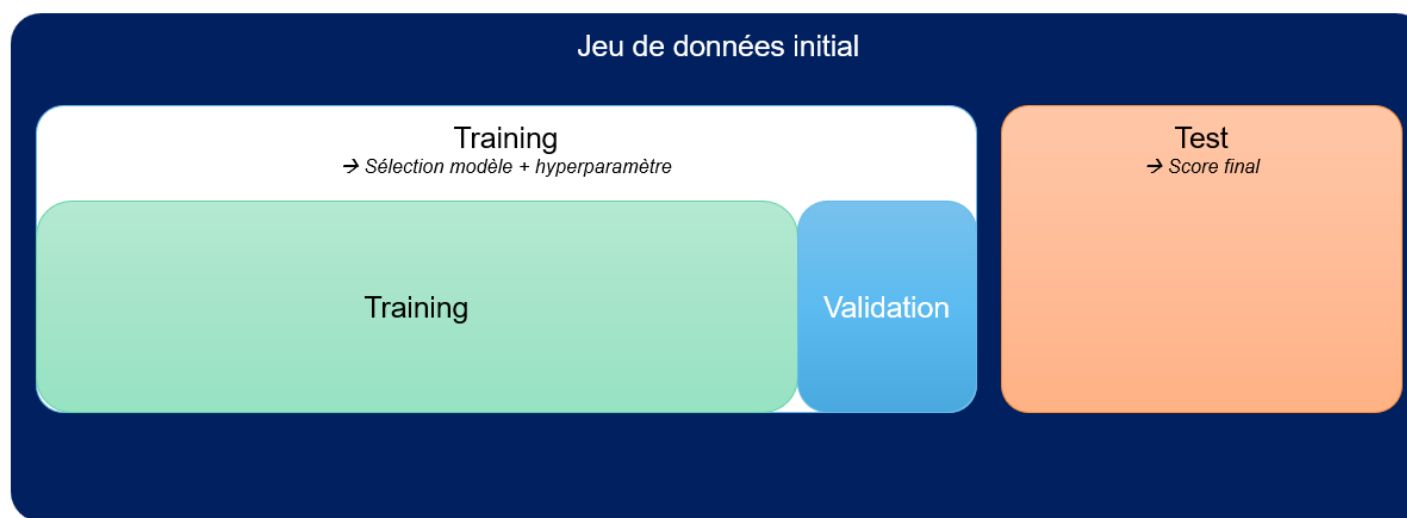
→ *Les projections sont trop précises sur l'historique (trop prudent ou trop risqué) mais diverge sur le futur*

Validation

Protocole de validation

Pourquoi séparer Train et Test ?

Mon modèle à 99% d'accuracy ? Oui mais sur quelle données ?



L'évaluation d'un modèle vise à mesurer ses performances de généralisation : *Mon modèle va-t-il être bon sur une distribution non vue durant l'entraînement ?*

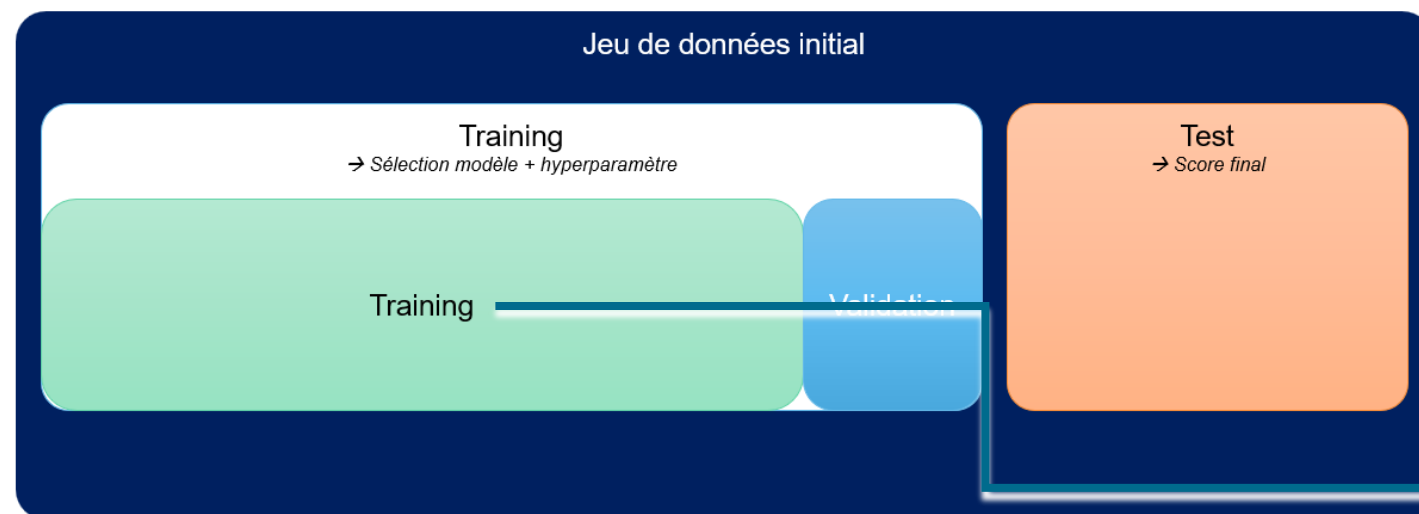
Le partitionnement de données en ensemble d'entraînement/validation/évaluation (train/val/test split) est une pratique qui vise à éviter le sur-apprentissage (= le modèle est très performant sur la base d'entraînement mais généralise mal).

Toutefois, il peut s'avérer qu'une seule division train/val/split soit insuffisante (risque de biais dans le choix du split) pour comparer plusieurs types de modèles (e.g. random forest, régression logistique, réseaux de neurones)

Protocole de validation

Validation croisée

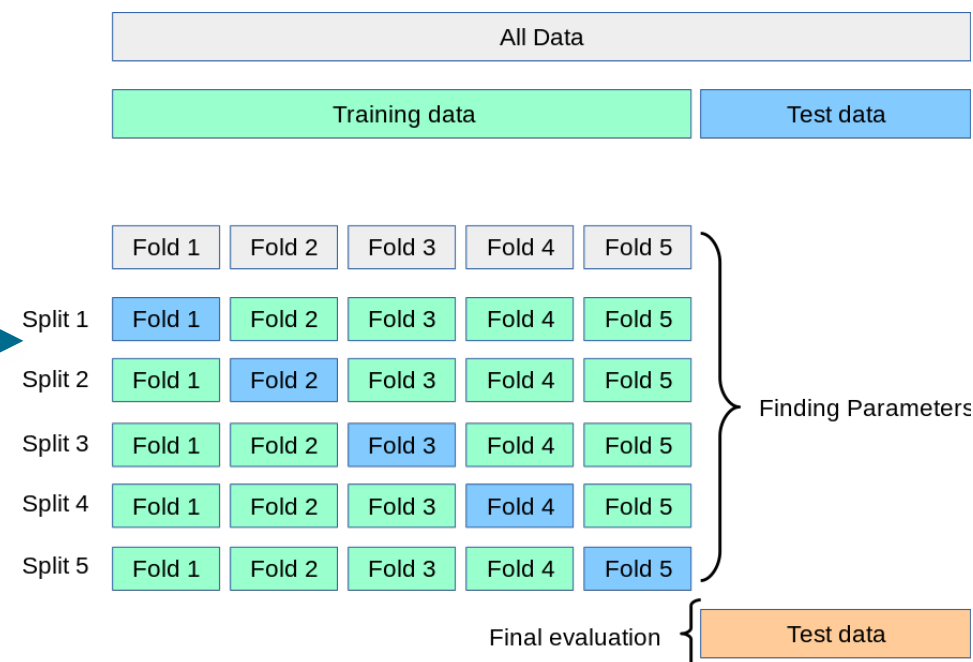
Multiplier les partitions du training set, pour multiplier les scénarios d'entraînement



⚠ Attention : la validation croisée permet d'évaluer des approches différentes sur un même dataset en comparant une performance moyenne suite aux entraînements faits par chaque approche. Elle ne permet pas d'obtenir la meilleure combinaison de paramètres pour une approche donnée (= feature engineering).

C'est une méthode de validation et non d'optimisation des paramètres.

Ex: On souhaite comparer une approche de Boosting vs une régression Logistique sur de la probabilité de rachat sur un portefeuille d'assurance vie (0= pas de rachat, 1= rachat). Pour cela, la validation croisée va permettre de comparer équitablement les deux approches sur le même dataset.



Protocole de validation

Variantes de validation croisée

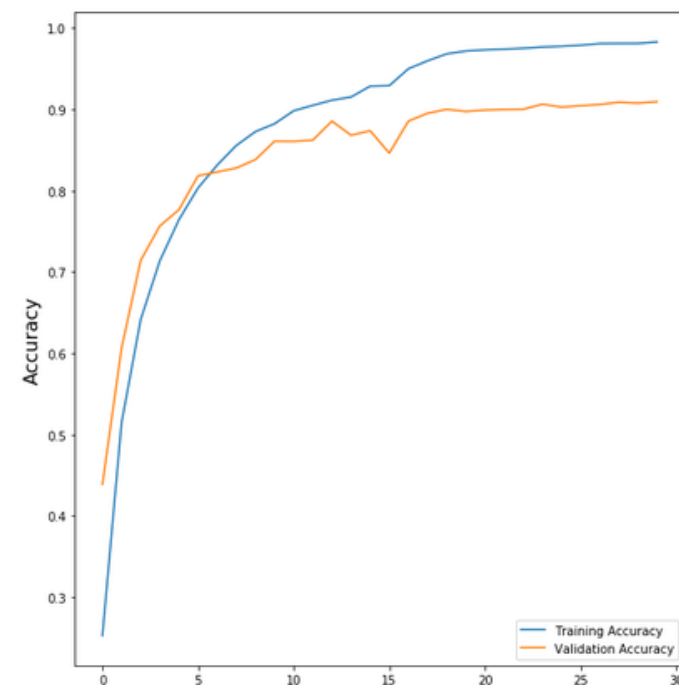
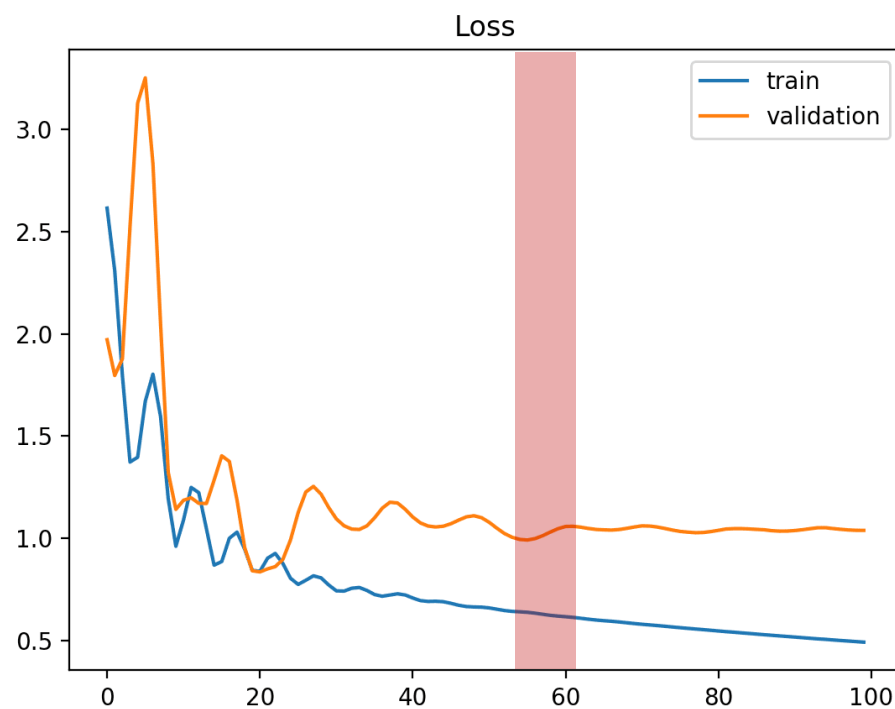
Variante	Quand l'utiliser	Particularité
K-Fold	Cas général (régression)	Standard
Stratified K-Fold	Classification (déséquilibrée)	Préserve les proportions de classes dans chaque fold
Leave-One-Out (LOO)	Très peu de données (<100)	K = n, très coûteux
Repeated K-Fold	Quand on veut plus de stabilité	Répète la CV avec différentes partitions
Group K-Fold	Données groupées (ex: même assuré)	Évite qu'un même groupe soit dans train ET val
TimeSeriesSplit	Données temporelles	Respecte l'ordre chronologique

Courbe d'apprentissage

Protocole de validation

Les courbes d'apprentissage

Outil de diagnostic en ML, les courbes d'apprentissage tracent le score du modèle en fonction du nombre d'échantillons vus par le modèle



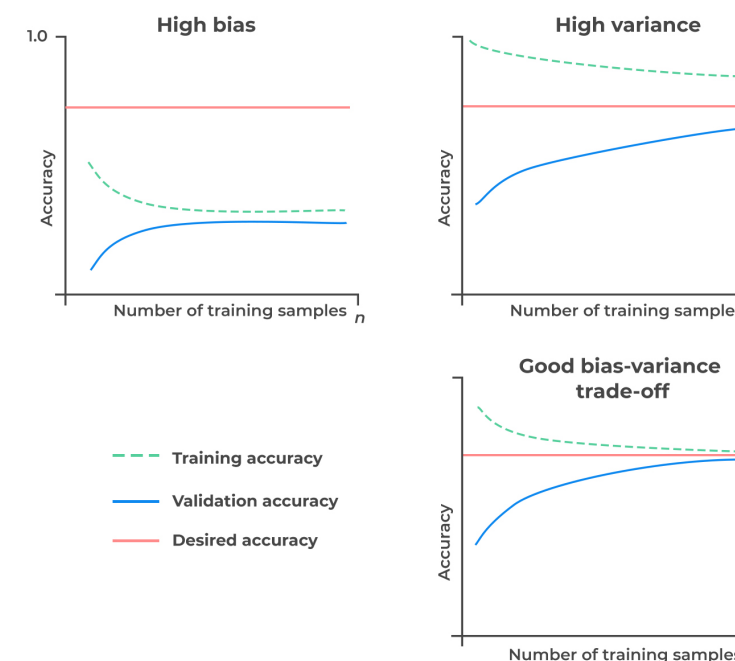
Protocole de validation

La courbe d'apprentissage: diagnostic

Diagnostic	Ce que vous voyez	Que faire ?
Biais élevé	Train et Val convergent bas	Modèle plus complexe, plus de features, moins de régularisation
Variance élevée	Train élevé, Val bien en dessous	Plus de données, régularisation, modèle plus simple, dropout
Bon compromis	Train et Val convergent haut, écart faible	Sweet spot
Pas assez de données	Val monte encore à la fin du graphe	Collecter plus de données (la courbe n'a pas convergé)



Learning Curves: Accuracy in Validation and Training Samples vs. Training



Bloc 3

Scikit-Learn

Scikit-Learn

Pourquoi sklearn?

Une interface unifiée pour le Machine Learning

```
# TOUT estimator sklearn suit ce pattern :

# 1. Instancier avec des hyperparamètres
model = SomeAlgorithm(param1=value1, param2=value2)

# 2. Entraîner sur des données (=Estimateur)
model.fit(X_train, y_train)

# 3. Prédire (=Predicteur)
y_pred = model.predict(X_test)

# ou Transformer (ACP, OneHotEncoder)
X_transformed = model.transform(X_test)

# 4. Évaluer
score = model.score(X_test, y_test)
```

Scikit-Learn

Conventions

```
# Les attributs appris pendant fit() se terminent par un underscore _
model.fit(X_train, y_train)

model.coef_                # Coefficients (régression linéaire)
model.intercept_           # Intercept
model.feature_importances_ # Importance des variables (arbres)
model.n_features_in_       # Nombre de features vues au fit
model.feature_names_in_    # Noms des features (si DataFrame)
model.classes_             # Classes (classification)

# Les hyperparamètres sont des arguments du constructeur
model = Ridge(alpha=1.0, fit_intercept=True)
model.get_params()          # Dictionnaire des hyperparamètres
model.set_params(alpha=0.5) # Modifier un hyperparamètre
```

Scikit-Learn

Exemple: Interchangeabilité des modèles

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.model_selection import cross_val_score

# Même interface pour TOUS ces modèles :
models = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(alpha=1.0),
    'Lasso': Lasso(alpha=0.1),
    'Decision Tree': DecisionTreeRegressor(max_depth=5),
    'Random Forest': RandomForestRegressor(n_estimators=100),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=100),
}

for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='r2')
    print(f"{name:25s} → R² = {scores.mean():.3f} ± {scores.std():.3f}")
```

Scikit-Learn

Pre-processing

Garbage In = Garbage out



Scikit-Learn

Pre-processing : le Scaling (normalisation/standardisation)

- Normaliser = redimensionner entre 0 et 1
- Standardiser = mise à l'échelle (Moyenne 0 écart-type 1)

Algorithme	Sensible au scaling ?	Pourquoi ?
Régression linéaire	Non*	Mais coefficients non comparables sans
Ridge / Lasso	OUI	La pénalisation traite toutes les features pareil
kNN	OUI	La distance euclidienne est dominée par les grandes échelles
SVM	OUI	Idem kNN
PCA	OUI	Maximise la variance (dominée par les grandes échelles)
Arbres (CART, RF, GBM)	Non	Splits basés sur des seuils, insensibles à l'échelle
Neural Networks	OUI	Gradient descent converge mal avec des échelles différentes

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# StandardScaler : moyenne=0, écart-type=1
#  $z = (x - \mu) / \sigma$ 
# → Bon par défaut, robuste

# MinMaxScaler : ramène dans [0, 1]
#  $x\_scaled = (x - x\_min) / (x\_max - x\_min)$ 
# → Quand on veut des bornes précises (images, réseaux de neurones)
```

Scikit-Learn

⚠ Piège n°1 : Scaler avant le split = DATA LEAKAGE

```
# ❌ MAUVAIS – Data Leakage !
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # fit sur TOUT le dataset (y compris test !)
X_train, X_test = train_test_split(X_scaled, ...)

# Pourquoi c'est grave ?
# Le scaler a vu les statistiques ( $\mu$ ,  $\sigma$ ) du test set pendant le fit.
# → Le modèle a une information "du futur" → score optimiste → mauvaise surprise en prod.
```

```
# ✅ CORRECT – Pas de leakage
X_train, X_test = train_test_split(X, ...)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit UNIQUEMENT sur train
X_test_scaled = scaler.transform(X_test)      # transform (sans re-fit) sur test
```

```
# ✅ ENCORE MIEUX – Pipeline (automatise le bon comportement)
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
pipe.fit(X_train, y_train) # Le scaler est fit UNIQUEMENT sur X_train
pipe.predict(X_test)      # Le scaler transform (sans re-fit) sur X_test
```

Scikit-Learn

Pre-processing : Encodage de variables catégorielles

Méthode	Quand l'utiliser	Exemple
OneHotEncoder	Catégoriel nominal (pas d'ordre)	Région, Marque véhicule
OrdinalEncoder	Catégoriel ordinal	Niveau de franchise (faible < moyen < élevé)
TargetEncoder	Haute cardinalité	Code postal (5000+ modalités)

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

# OneHotEncoder crée une colonne binaire par modalité
# "Region" = {Nord, Sud, Est, Ouest}
# → 4 colonnes : is_Nord, is_Sud, is_Est, is_Ouest

ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
```

Scikit-Learn

⚠ Piège n°2 : pd.get_dummies

```
# ❌ PROBLÈME avec pd.get_dummies
X_train_encoded = pd.get_dummies(X_train) # Crée colonnes pour chaque catégorie vue
X_test_encoded = pd.get_dummies(X_test)   # Peut créer des colonnes DIFFÉRENTES !

# Si le train a Region={Nord, Sud, Est} et le test a Region={Nord, Ouest}
# → X_train_encoded a 3 colonnes, X_test_encoded en a 2 → ERREUR !
```

```
# ✅ OneHotEncoder gère ça proprement
ohe = OneHotEncoder(handle_unknown='ignore')
ohe.fit(X_train[['Region']]) # Apprend les catégories du train
ohe.transform(X_test[['Region']]) # Catégories inconnues → vecteur nul
```

```
# ✅ ENCORE MIEUX – Pipeline (automatise le bon comportement)
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
pipe.fit(X_train, y_train) # Le scaler est fit UNIQUEMENT sur X_train
pipe.predict(X_test)       # Le scaler transform (sans re-fit) sur X_test
```


Scikit-Learn

Pre-processing : Gestion des valeurs manquantes

```
from sklearn.impute import SimpleImputer, KNNImputer

# SimpleImputer : remplace par une statistique (mean, median, most_frequent, constant)
imputer = SimpleImputer(strategy='median')

# KNNImputer : impute en se basant sur les k plus proches voisins
imputer = KNNImputer(n_neighbors=5)
```

⚠ PIÈGE N°3 : Imputer avant le split = DATA LEAKAGE

```
# ❌ MAUVAIS
X['age'].fillna(X['age'].mean(), inplace=True) # Moyenne calculée sur tout le dataset
X_train, X_test = train_test_split(X, ...)

# ✅ CORRECT (via Pipeline)
pipe = make_pipeline(SimpleImputer(strategy='median'), Ridge())
pipe.fit(X_train, y_train) # L'imputer calcule la médiane UNIQUEMENT sur le train
```

Scikit-Learn

Les Pipelines

Un framework pour éviter les erreurs classiques

Le pipeline concentre les étapes de preprocessing et de modélisation dans le même objet Python. Des règles sont ensuite appliquées pour éviter les pièges habituels:

- Pas de data leakage
- Reproductibilité: tout le preprocessing + le modèle en un seul objet
- Déploiement facile (sauvegarde du modèle en pickle)
- Cross-validation propre : le preprocessing est refait à chaque fold

```
from sklearn.pipeline import Pipeline, make_pipeline

# Méthode 1 : Pipeline explicite (avec noms)
pipe = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('model', Ridge(alpha=1.0))
])

# Méthode 2 : make_pipeline (noms automatiques)
pipe = make_pipeline(StandardScaler(), Ridge(alpha=1.0))

# Utilisation : exactement comme un modèle simple !
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
score = pipe.score(X_test, y_test)
```

Bloc 4

Régression Linéaire

Régression linéaire

La même régression, deux philosophies

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \boldsymbol{\beta}^T \mathbf{x}$$

Formulation matricielle:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta} \quad \text{model.predict(X)}$$

Fonction objective (OLS / Ordinary Least Square):

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 = \arg \min_{\boldsymbol{\beta}} \sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2 \quad \text{model.fit()}$$

Solution analytique:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

La regression linéaire est adaptée si:

- Vous devez expliquer chaque prediction précisément
- Vous avez peu de données
- Vous voulez une reference avant de tester des modèles complexes (baseline)
- Vos features sont très bien construites
- Les effets sont monotones et additifs

Régression linéaire

La même régression, deux philosophies

	statsmodels (Stats)	sklearn (ML)
Code	sm.OLS(y, X).fit()	LinearRegression().fit(X, y)
Ce qu'on obtient	p-values, R ² , AIC, résidus, CI	coef_, intercept_, score()
Validation	F-test, t-tests, Durbin-Watson	Cross-validation
Philosophie	"Ce coefficient est-il significatif ?"	"Ce modèle prédit-il bien ?"
Usage	Rapport d'étude, publication	Pipeline, production

```
# Vision statistique (statsmodels)
import statsmodels.api as sm
X_sm = sm.add_constant(X_train)
model_sm = sm.OLS(y_train, X_sm).fit()
print(model_sm.summary()) # p-values, CI, R2, etc.

# Vision ML (sklearn)
from sklearn.linear_model import LinearRegression
model_sk = LinearRegression()
model_sk.fit(X_train, y_train)
print(f"R2 test : {model_sk.score(X_test, y_test):.4f}") # Généralisation
```

Régularisation : Ridge, Lasso, ElasticNet

Pourquoi régulariser ?

La régression n'a pas de contrôle de sa complexité
→ *Plus vous avez de features (surtout corrélées)*
→ *Plus les coefficients peuvent exploser en magnitude*

Ridge (L2) – Pénalité quadratique

$$\hat{\boldsymbol{\beta}}_{Ridge} = \arg \min_{\boldsymbol{\beta}} \underbrace{\sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2}_{\text{Fidélité aux données}} + \underbrace{\alpha \sum_{j=1}^p \beta_j^2}_{\text{Pénalité L2}}$$

- **Effet** : Réduit la magnitude de TOUS les coefficients (sans les annuler)
- **Quand l'utiliser** : Multicolinéarité, stabilisation des coefficients

Régularisation : Ridge, Lasso, ElasticNet

Pourquoi régulariser ?

La régression n'a pas de contrôle de sa complexité
→ *Plus vous avez de features (surtout corrélées)*
→ *Plus les coefficients peuvent exploser en magnitude*

Lasso (L1) – Pénalité absolue

$$\hat{\boldsymbol{\beta}}_{Lasso} = \arg \min_{\boldsymbol{\beta}} \sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2 + \alpha \sum_{j=1}^p |\beta_j|$$

- **Effet** : Réduit certains coefficients exactement à 0 → Sélection de variables
- **Quand l'utiliser** : Quand vous pensez que seules certaines features sont pertinentes

Régularisation : Ridge, Lasso, ElasticNet

Pourquoi régulariser ?

La régression n'a pas de contrôle de sa complexité
→ *Plus vous avez de features (surtout corrélées)*
→ *Plus les coefficients peuvent exploser en magnitude*

ElasticNet – Combinaison L1 + L2

$$\hat{\boldsymbol{\beta}}_{EN} = \arg \min_{\boldsymbol{\beta}} \sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2 + \alpha \left[\rho \sum_{j=1}^p |\beta_j| + \frac{(1 - \rho)}{2} \sum_{j=1}^p \beta_j^2 \right]$$

- **Effet** : Combinaison des avantages de Ridge et Lasso
- **Quand l'utiliser** : Beaucoup de features (selection de variable) et certaines sont groupées/corrélées (stabilisation des coefficients)

Régularisation : Ridge, Lasso, ElasticNet

Pourquoi régulariser ?

Ridge (L2) – Pénalité quadratique

$$\hat{\boldsymbol{\beta}}_{Ridge} = \arg \min_{\boldsymbol{\beta}} \underbrace{\sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2}_{\text{Fidélité aux données}} + \underbrace{\alpha \sum_{j=1}^p \beta_j^2}_{\text{Pénalité L2}}$$

Ridge (L2) – Pénalité quadratique

$$\hat{\boldsymbol{\beta}}_{Lasso} = \arg \min_{\boldsymbol{\beta}} \sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2 + \alpha \sum_{j=1}^p |\beta_j|$$

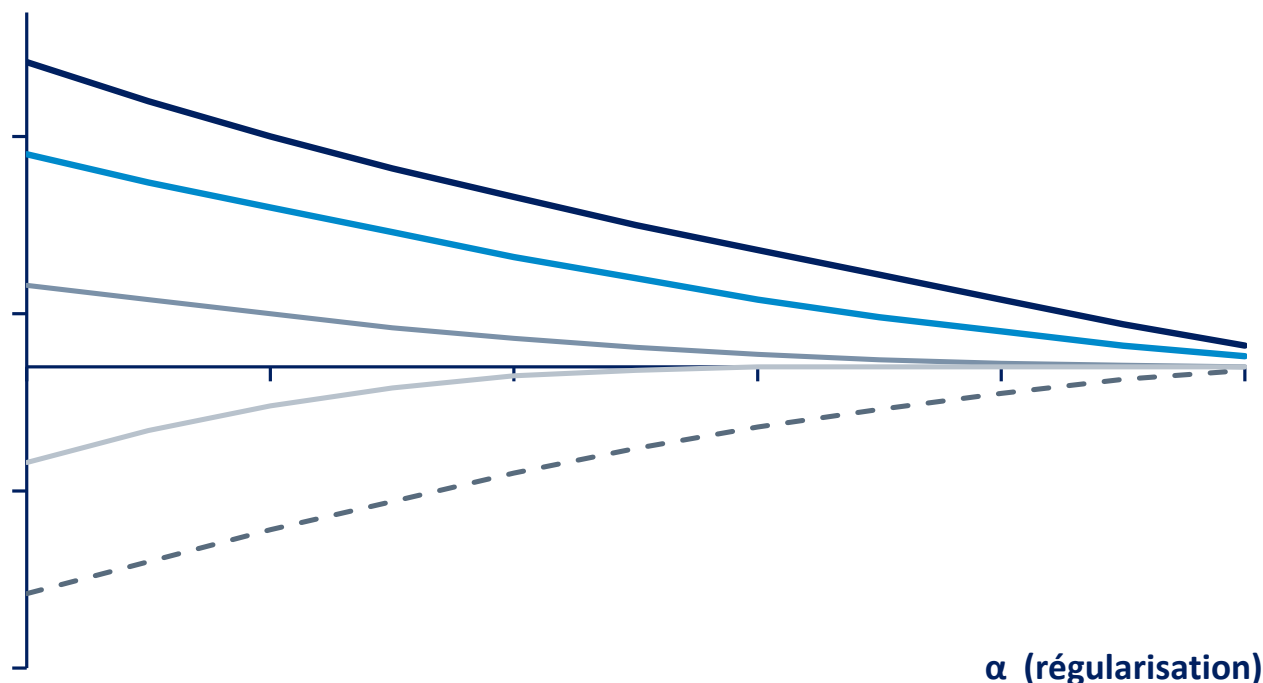
ElasticNet – Combinaison L1 + L2

$$\hat{\boldsymbol{\beta}}_{EN} = \arg \min_{\boldsymbol{\beta}} \sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2 + \alpha \left[\rho \sum_{j=1}^p |\beta_j| + \frac{(1-\rho)}{2} \sum_{j=1}^p \beta_j^2 \right]$$

Régularisation : Ridge, Lasso, ElasticNet

Pourquoi régulariser ? → Lien avec le dilemme biais-variance

Coefficients β



F1–F2 importantes F3 peu utile F4 bruit F5 corrélée à F1

$\alpha = 0$

$\alpha = \infty$

sweet spot • trouvé par validation croisée

Critère	$\alpha = 0$ pas de régul.	$\alpha = \infty$ régul. maximale
Biais	Faible	Élevé (prédit la moyenne)
Variance	Élevée (instable)	Faible (très stable)
Complexité	Maximale	Minimale
Modèle	OLS classique	Constante

Régularisation : Ridge, Lasso, ElasticNet

En pratique avec Sklearn

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet, RidgeCV, LassoCV

# Ridge avec alpha fixe
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)

# Ridge avec sélection automatique de alpha par CV
ridge_cv = RidgeCV(alphas=np.logspace(-3, 5, 100), cv=5)
ridge_cv.fit(X_train_scaled, y_train)
print(f"Meilleur alpha : {ridge_cv.alpha_:.4f}")

# Lasso avec sélection automatique
lasso_cv = LassoCV(cv=5, random_state=42)
lasso_cv.fit(X_train_scaled, y_train)
print(f"Meilleur alpha : {lasso_cv.alpha_:.4f}")
print(f"Features sélectionnées : {np.sum(lasso_cv.coef_ != 0)} / {X_train_scaled.shape[1]}")

# ⚠ IMPORTANT : Ridge et Lasso nécessitent un scaling !
# Les pénalités traitent tous les coefficients de la même façon.
# Si les features ont des échelles différentes, celles avec les plus
# grandes valeurs seront pénalisées disproportionnellement.
```

Régularisation : Ridge, Lasso, ElasticNet

Application actuarielle : sélection de variables en tarification

Des dizaines de variables candidates

- Âge
- Ancienneté
- BonusMalus
- Zone
- Véhicule
- Profession
- Kilométrage
- Énergie
- Usage
- Couleur

... et beaucoup d'autres

LASSO

$\beta = 0$ écartées
 α choisi par CV

$\beta \neq 0$

- BonusMalus
- Ancienneté
- Zone
- Type de véhicule

4 variables retenues

GLM final

Calibrer

Interpréter

Généraliser

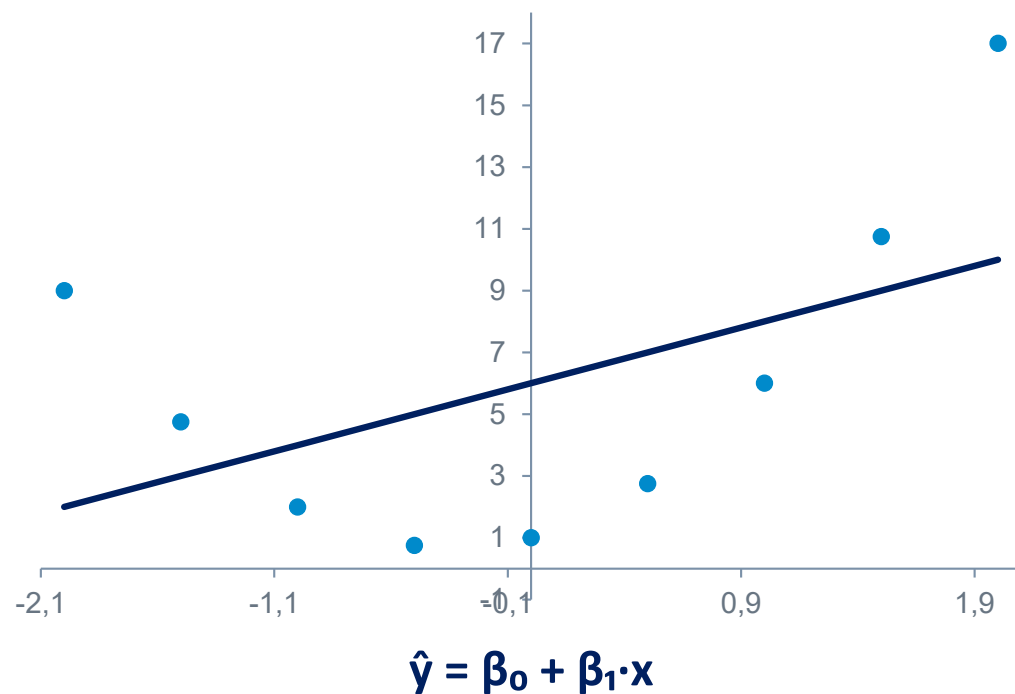
moins de variables
moins d'overfitting

Lasso pour sélectionner → **GLM** pour calibrer et expliquer

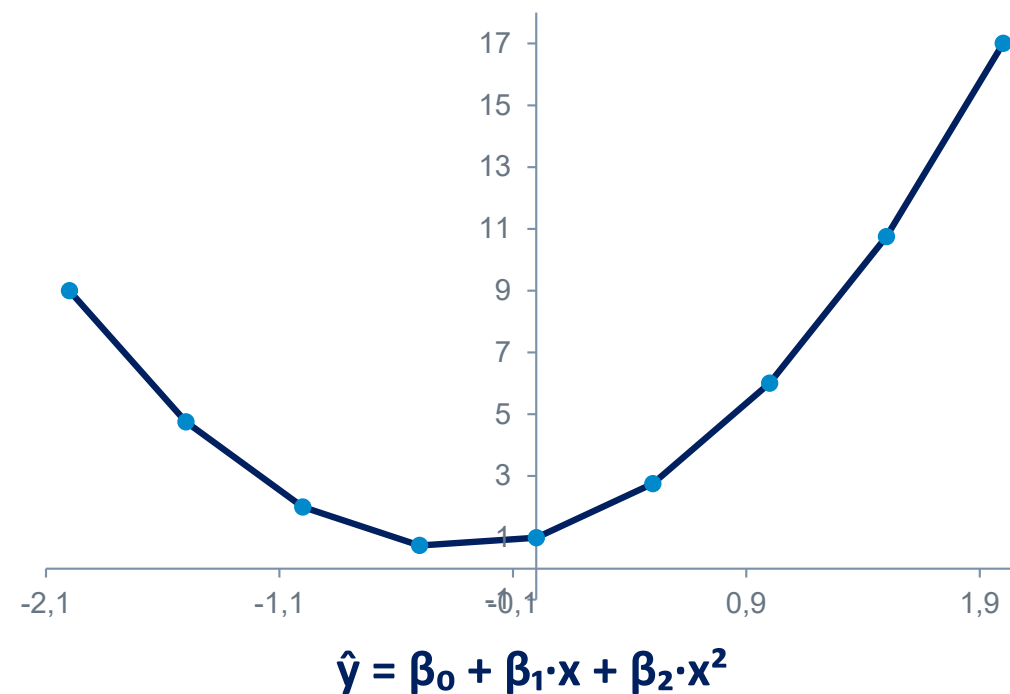
Feature Engineering & Polynomial Features

Le pouvoir du feature engineering

× x seul : la droite manque la courbure



✓ $x + x^2$: la parabole devient accessible



La courbe est non-linéaire en x , mais le modèle reste linéaire en β .

Feature Engineering & Polynomial Features

PolynomialFeatures : créer de nouvelles colonnes

Une observation

$X = [2, 3]$



degree = 2

x_1	x_2	x_1^2	$x_1 \cdot x_2$	x_2^2
2	3	4	6	9

API essentielle

```
PolynomialFeatures(degree=2,  
include_bias=False)
```

```
X_poly = poly.fit_transform(X)
```

Risque d'explosion combinatoire

2 → **5** features

10 → **286** au degré 3

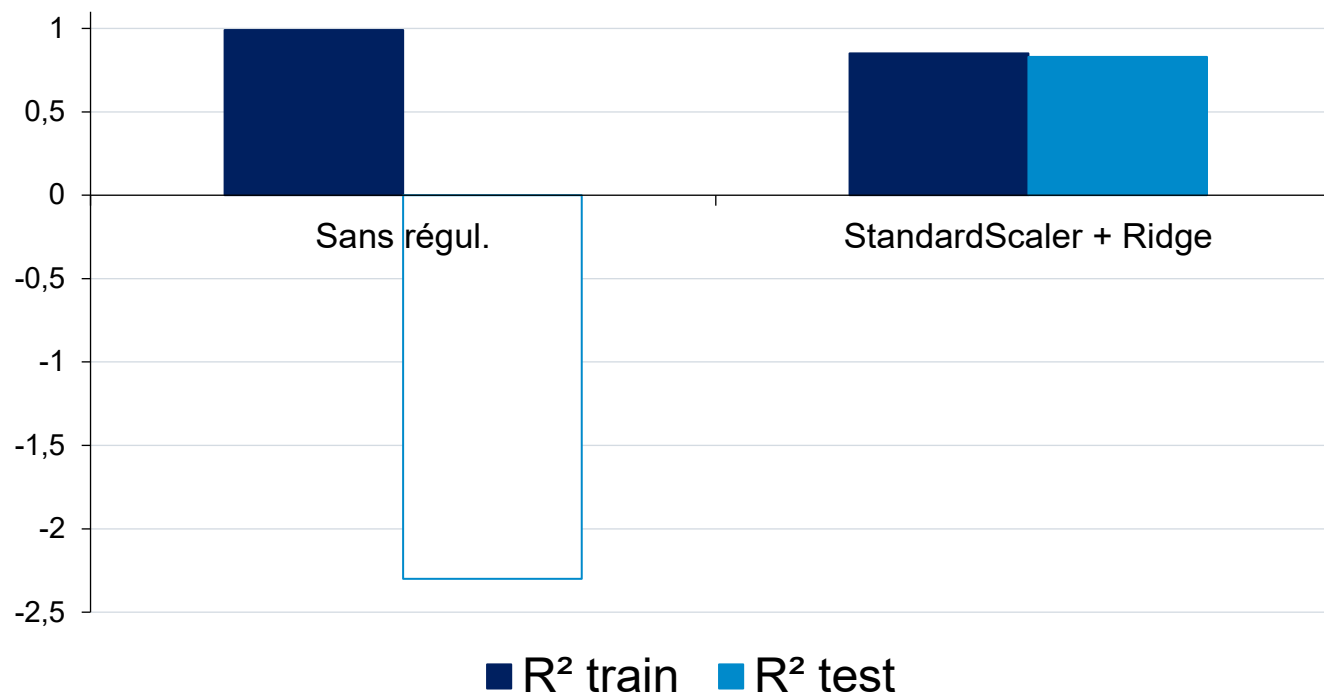
50 → **1 326** au degré 2

Règle pratique : **PolynomialFeatures + Ridge / Lasso**

Feature Engineering & Polynomial Features

Degré élevé : la régularisation fait la différence

R^2 : le test révèle immédiatement l'overfit



× Pipeline fragile

```
PolynomialFeatures(10)
```

```
LinearRegression()
```

→ train 0.99 · test -2.30

✓ Pipeline maîtrisé

```
PolynomialFeatures(10)
```

```
StandardScaler()
```

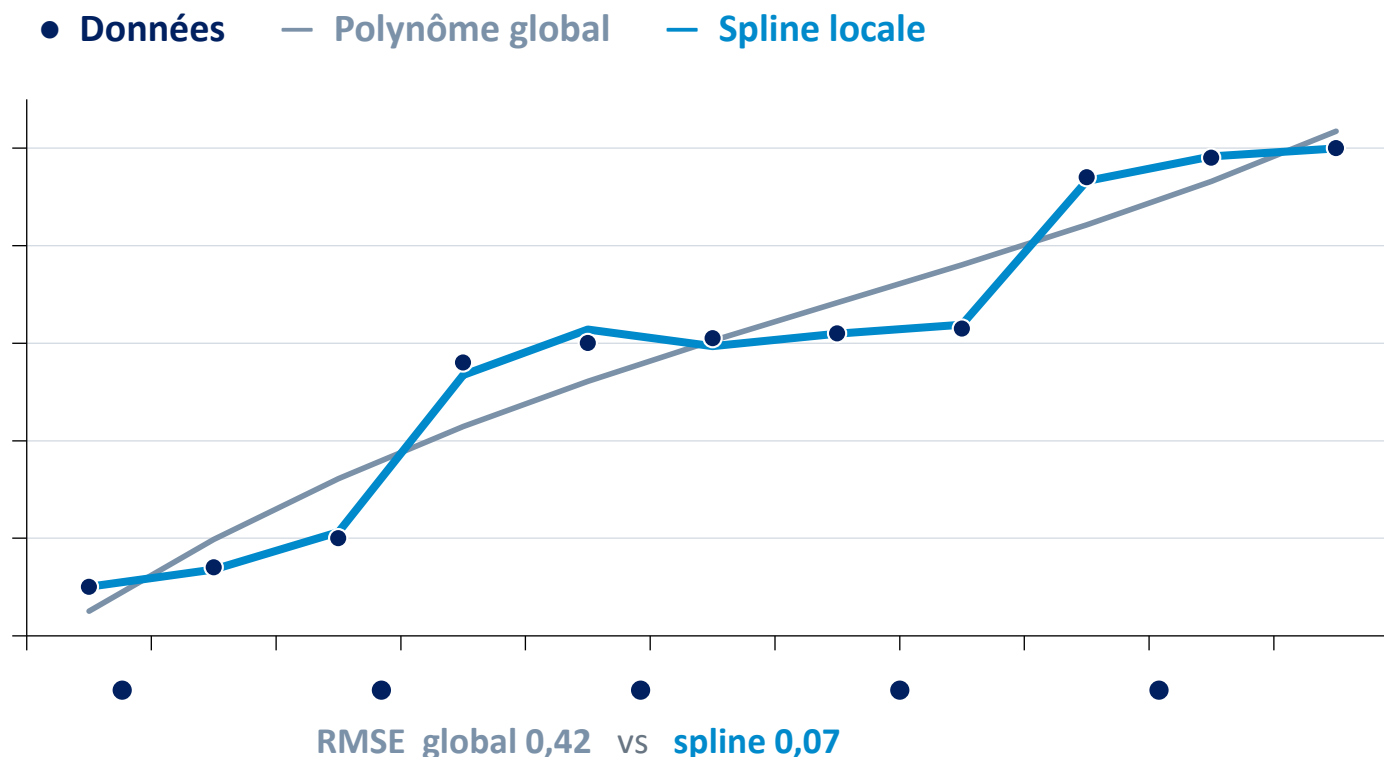
```
Ridge(alpha=1.0)
```

→ train 0.85 · test 0.83

Non-linéaire + régularisé = flexible sans mémoriser le bruit

Feature Engineering & Polynomial Features

SplineTransformer : une alternative plus locale et stable



Pourquoi les splines ?

- **Locale**
agit par morceaux
- **Stable aux bords**
moins d'oscillations
- **Interprétable**
courbe lisse et contrôlée

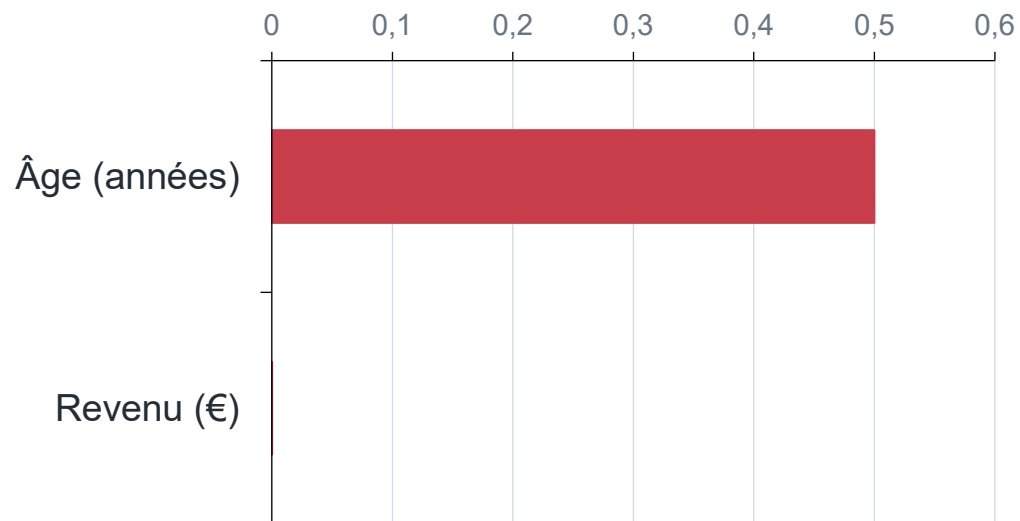
En assurance : **splines / GAM** reprennent le même principe, avec plus de robustesse.

```
SplineTransformer(degree=3, n_knots=5)  
pipe_spline = make_pipeline(spline, Ridge(alpha=1.0))
```


Interprétation & diagnostic

Attention au scaling pour interpréter les coefficients

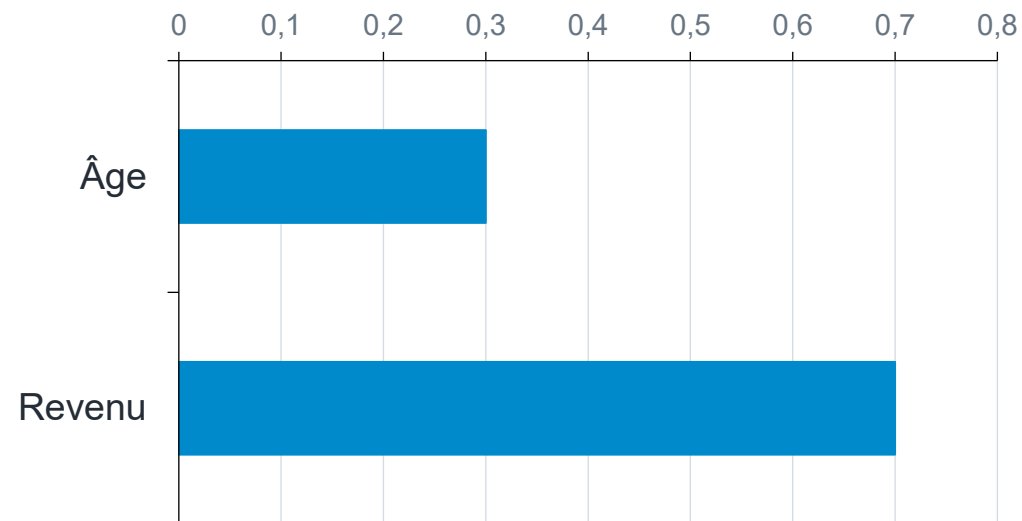
Sans scaling · unités incompatibles



$\beta_{\text{âge}} = 0,5$ vs $\beta_{\text{revenu}} = 0,0001$

+1 unité de $x_j \rightarrow +\beta_j$ unités de y

Avec StandardScaler · même échelle



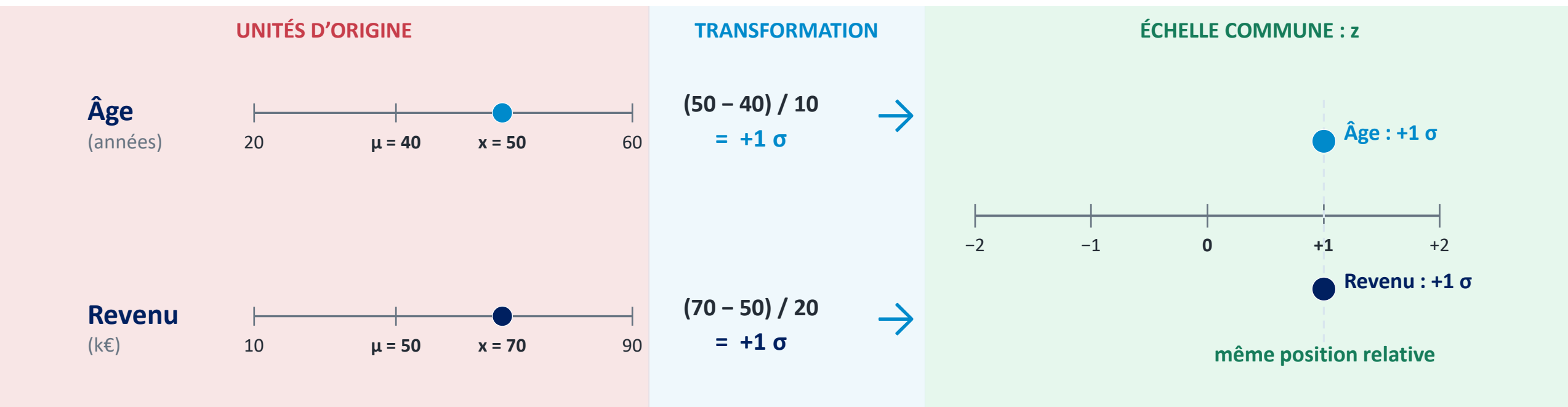
$\beta_{\text{âge}} = 0,3$ vs $\beta_{\text{revenu}} = 0,7$

+1 écart-type de $x_j \rightarrow +\beta_j$ unités de y

Comparer $|\beta|$? **Scaling obligatoire.** Ici, le revenu a le plus d'impact.

Interprétation & diagnostic

Le scaling remplace les unités par une distance à la moyenne



$$z_j = (x_j - \mu_{j,\text{train}}) / \sigma_{j,\text{train}} \rightarrow \text{moyenne } 0 \cdot \text{écart-type } 1$$

Comparer à armes égales : **le même déplacement relatif** (un écart-type) plutôt que 1 an contre 1 €.

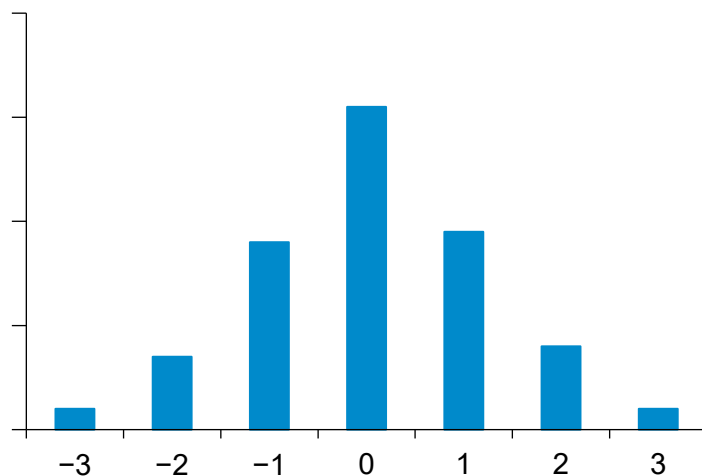
μ et σ sont appris sur le TRAIN uniquement · $\sigma_j \neq 0$

Interprétation & diagnostic

Diagnostic par les résidus : trois questions, trois graphiques

```
residuals = y_test - y_pred
```

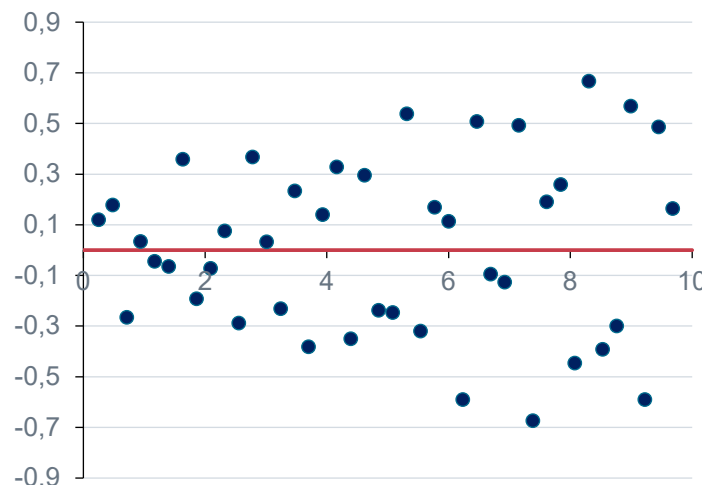
1 Les résidus sont-ils centrés ?



✓ autour de 0 · symétrique

```
# 1. Distribution des résidus  
plt.hist(residuals, bins=50)
```

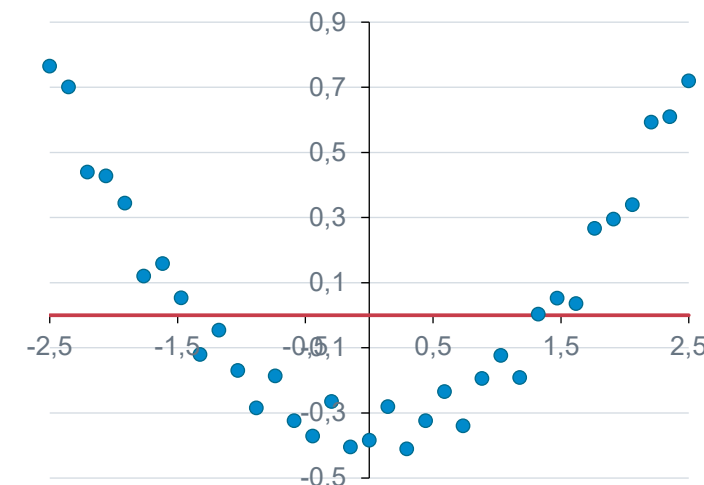
2 Y a-t-il une structure ?



! entonnoir → variance non constante

```
# 2. Résidus vs prédictions  
plt.scatter(y_pred, residuals)
```

3 Quelle feature est coupable ?



! courbe → effet non capté

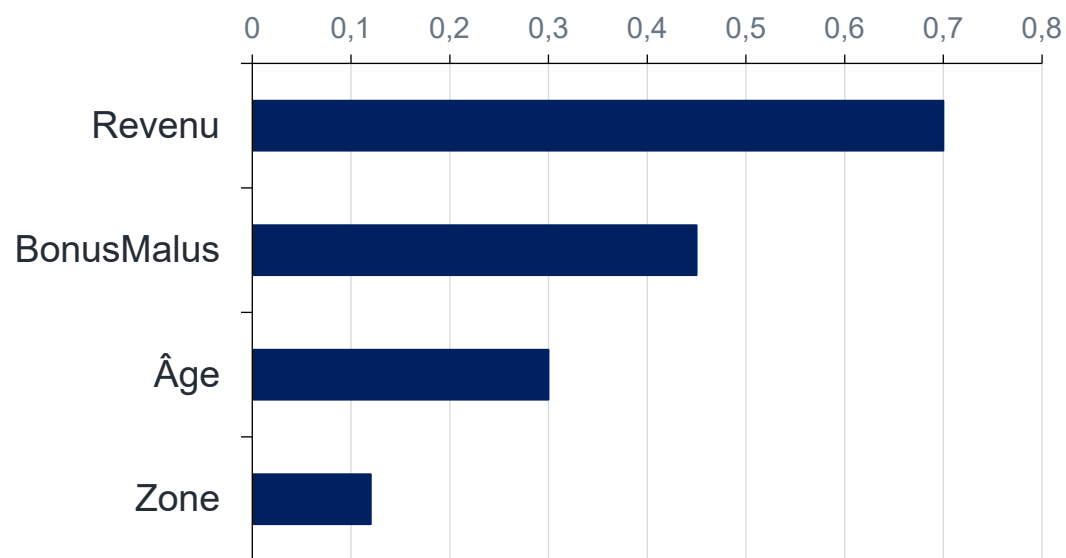
```
# 3. Résidus vs features  
for feature in X.columns:  
    plt.scatter(X_test[feature], residuals)
```

En ML, les résidus servent à détecter **patterns**, **segments faibles** et **biais structurels** — pas à tester la normalité.

Interprétation & diagnostic

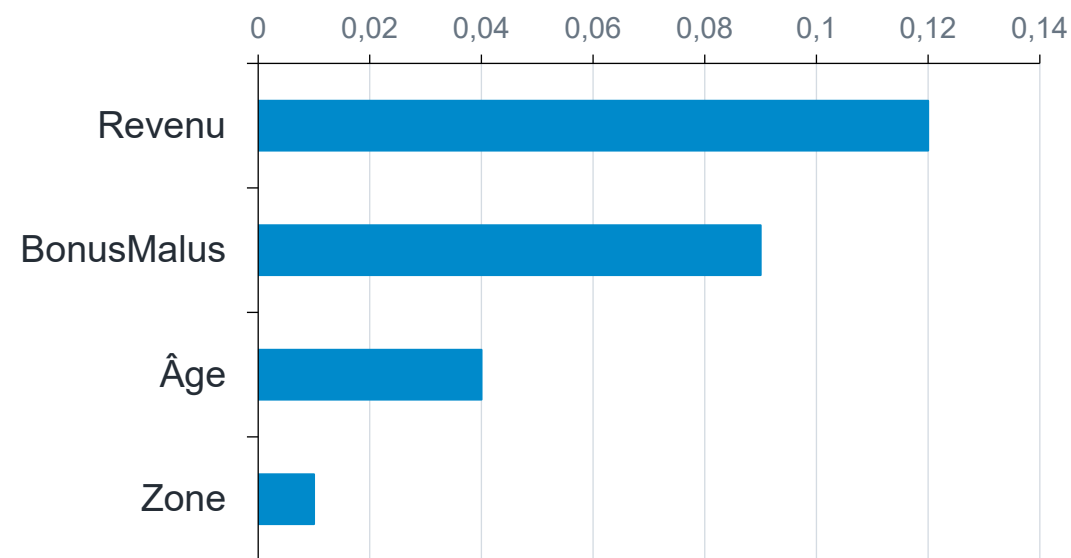
Mesurer l'importance des features : deux méthodes complémentaires

1 Magnitude des coefficients $|\beta|$



Rapide · modèle linéaire · après scaling

2 Permutation importance



Mélanger 1 feature → mesurer la baisse de score

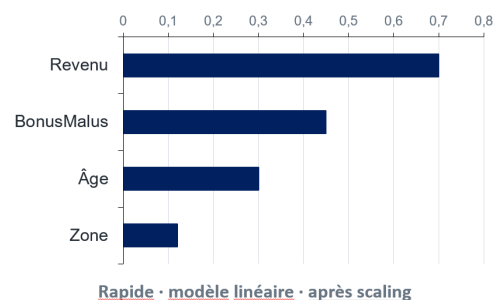
Lecture rapide : $|\beta|$ standardisés · Validation robuste : permutation sur le jeu de test

La permutation fonctionne avec n'importe quel modèle.

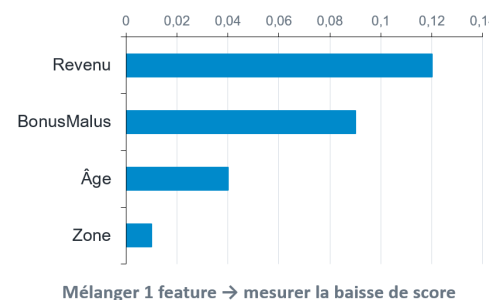
Interprétation & diagnostic

Mesurer l'importance des features : deux méthodes complémentaires

1 Magnitude des coefficients $|\beta|$



2 Permutation importance



```
# Méthode 1 : Magnitude des coefficients (après scaling)
```

```
importance = pd.Series(  
    np.abs(model.coef_),  
    index=feature_names  
) .sort_values(ascending=False)
```

```
# Méthode 2 : Permutation importance (plus robuste, marche pour tout modèle)
```

```
from sklearn.inspection import permutation_importance  
result = permutation_importance(model, X_test, y_test, n_repeats=10)  
importance_perm = pd.Series(  
    result.importances_mean,  
    index=feature_names  
) .sort_values(ascending=False)
```

Mise en perspective

Le fil rouge de la journée : comprendre → diagnostiquer → construire → pratiquer

BLOC 1

1

De la statistique au ML

- Philosophie
- Paradigmes
- Choisir le bon cadre

COMPRENDRE

BLOC 2

2

Le cœur du ML : généraliser

- Biais–variance
- Cross-validation
- Learning curves

DIAGNOSTIQUER

BLOC 3

3

La boîte à outils : scikit-learn

- Pipelines
- Preprocessing
- Pièges courants

CONSTRUIRE

BLOC 4

4

Application : régression + régularisation

- Ridge & Lasso
- Feature engineering
- Interprétation

PRATIQUER

Un projet ML solide suit le même ordre : **cadrer** → **mesurer** → **industrialiser** → **agir**

Mise en perspective

Un projet ML réaliste commence par le métier — pas par l’algorithme



Commencer simple donne **une référence**, **une compréhension** et un **time-to-value rapide**.

Mise en perspective

Les 10 commandements : mesurer juste, construire robuste

Mesurer juste

- 1 Commencer simple** une baseline avant la complexité
- 2 Sanctuariser le test** une seule évaluation finale
- 4 Valider par CV** plus robuste qu'un seul split
- 5 Aligner la métrique** sur la décision métier
- 6 Tracer les courbes** pour lire biais et variance

Construire robuste

- 3 Assembler un Pipeline** préprocessing + modèle
- 7 Traquer le leakage** temporel et dans les features
- 8 Régulariser** pour contrôler le compromis
- 9 Douter du score parfait** trop beau = signal d'alerte
- 10 Documenter clairement** résultats, limites, monitoring

Le principe commun : **protéger la généralisation** avant d'optimiser la performance apparente.

Mise en perspective

Questions bonus: cinq réponses à savoir formuler simplement

- | | | |
|-----------|--|--|
| Q1 | Biais–variance en 2 phrases | Biais = erreur systématique ; variance = sensibilité au train. Chercher le minimum d’erreur de généralisation. |
| Q2 | Ridge vs Lasso | L2 réduit tous les coefficients ; L1 peut en annuler certains et sélectionner des variables. |
| Q3 | Pourquoi la cross-validation ? | Elle moyenne plusieurs splits : estimation plus robuste et variabilité visible. |
| Q4 | Exemple de data leakage en assurance | Utiliser en feature une information <i>causée</i> par l’événement cible (ou indisponible au moment réel de la prédiction). |
| Q5 | « Pourquoi ce client paie plus cher ? » | Il faut un modèle interprétable. GLM ou arbre simple ; si GBM, ajouter une explication locale de type SHAP. |